

Absolutely, Dude. Here is the **copy-paste-ready database.md**, aligned with the accepted PRD and **techstack.md**.

Centralized Real-Time Queue Management Platform

Database Specification — `database.md`

1. Document Information

| Field | Details |

|---|---|

| Project | Centralized Real-Time Queue Management Platform |

| Document | database.md |

| Version | 1.0 |

| Status | Proposed |

| Database | PostgreSQL |

| Architecture | Multi-Tenant Relational Database |

2. Database Objective

PostgreSQL will be used as the primary persistent database for the application.

The database must support:

- Multiple tenants/organizations.
- Tenant data isolation.

- Administrators.
- Queue operators.
- End users.
- Queues.
- Queue entries.
- Dynamic queue positions.
- Queue status changes.
- Admin queue reordering.
- User cancellation.
- Queue completion.
- Estimated waiting time.
- Notification records.
- Queue event history.
- Audit records.

The database must remain generic and must not contain industry-specific concepts such as:

- Restaurant tables.

- Hospital rooms.

- Doctors.

- Government counters.

- Service desks.

- Resource assignment.

The core database model is based on:

``text

Tenant



Queue



Queue Entry



Queue Events



Notifications

3. Multi-Tenant Database Strategy

The proposed database strategy is:

One PostgreSQL Database



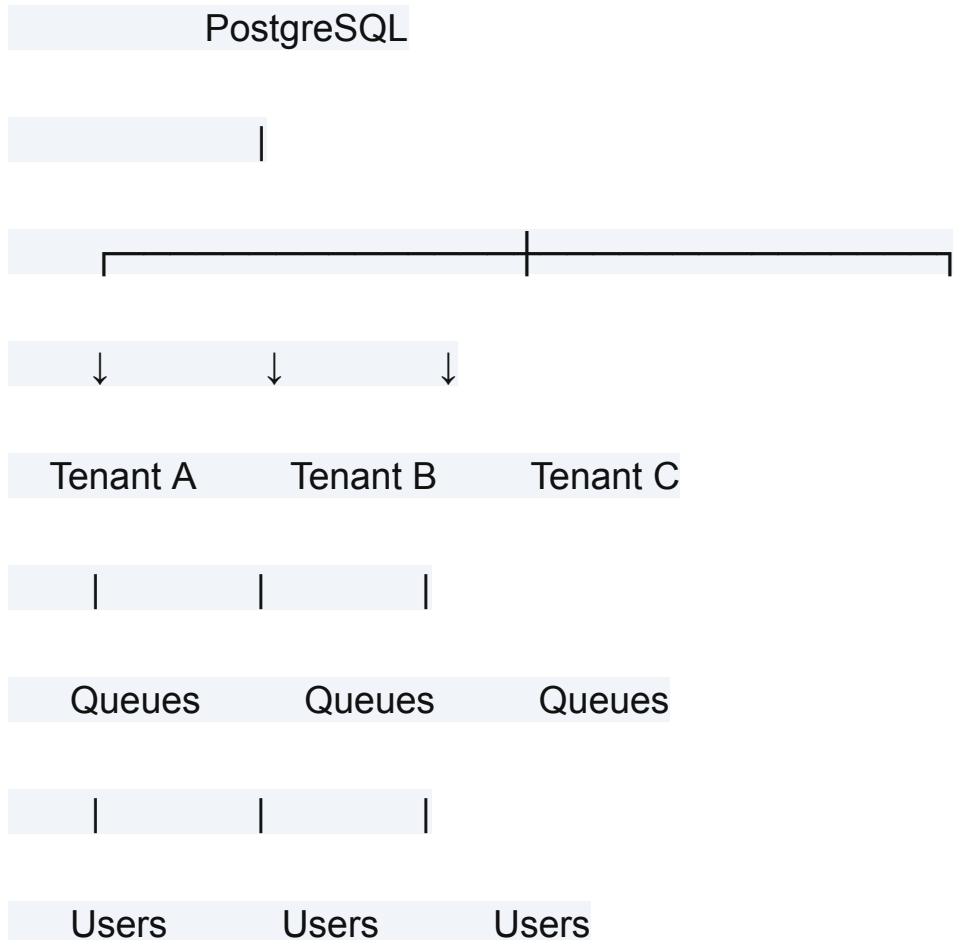
Shared Schema



Tenant Isolation

Each organization is represented as a tenant.

Conceptually:



The final implementation must ensure that one tenant cannot access another tenant's data.

4. Tenant Isolation

Tenant isolation is a critical security requirement.

Tenant-owned records should contain a `tenant_id` or equivalent

relationship where appropriate.

Example:

tenant_id = A



Queue A

Queue Entry A

Notification A

User A

Tenant A must never be able to access:

tenant_id = B

through an API, database query, or frontend request.

Tenant isolation must be enforced in the backend data-access layer.

The frontend must never be relied upon for tenant isolation.

5. Core Entity Relationship

The high-level relationship is:

Tenant

|

+---- Users

|

+---- Queues

|

+---- Queue Entries

|

+---- Queue Events

|

+---- Notifications

|

+---- Audit Events

6. Core Tables

The initial database will contain the following major entities:

tenants

users

queues

queue_entries

queue_events

notifications

notification_deliveries

audit_events

Additional supporting tables may be introduced if required by the final architecture.

7. Tenants Table

Purpose

Stores organizations using the centralized queue platform.

Example:

Tenant A

Tenant B

Tenant C

Suggested Fields

Field	Purpose
id	Unique tenant identifier
name	Organization name
slug	Unique tenant identifier for URLs/internal references
status	Tenant status
created_at	Creation timestamp

update_date	Last modification timestamp
-------------	-----------------------------

Possible tenant statuses:

ACTIVE

SUSPENDED

INACTIVE

The final status model may be adjusted during implementation.

8. Users Table

Purpose

Stores authenticated users who operate or administer the platform.

Users may belong to a tenant.

Suggested Fields

Field	Purpose
-------	---------

id	Unique user identifier
tenant_id	Tenant relationship
name	User name
email	Login/contact email where applicable
phone	Phone number where applicable
password_hash	Secure password representation if password authentication is used
role	User role
status	Account status
created_at	Creation timestamp
updated_at	Last modification timestamp

ed_at

Passwords must never be stored as plain text.

9. User Roles

The initial roles are:

PLATFORM_ADMIN

ORGANIZATION_ADMIN

QUEUE_OPERATOR

End users do not necessarily require permanent platform accounts.

10. Platform Admin

A platform administrator operates at the platform level.

Potential access includes:

- Tenant management.
- Platform configuration.
- Platform administration.
- System-level operations.

The exact permissions will be finalized in the application authorization

design.

11. Organization Admin

The Organization Admin operates within a tenant.

The Organization Admin can:

- Manage queues.
 - Generate QR codes.
 - Modify queue positions.
 - Manage authorized operators.
 - Configure relevant queue settings.
 - View tenant-specific queue information.
-

12. Queue Operator

Queue operators manage active queues.

They can perform permitted queue operations such as:

- View queue.
- Call/serve the next user where applicable.
- Mark an entry DONE.
- Handle cancellation/no-show states where permitted.
- Perform other approved queue actions.

Operators must only access queues they are authorized to manage.

13. End Users

End users are people joining a queue.

They do not necessarily require records in the `users` table.

Their queue-related information should primarily be represented through `queue_entries`.

Example:

User scans QR



Queue Entry Created



Queue Entry stores required information

This avoids forcing every temporary queue participant to create a permanent account.

14. Queues Table

Purpose

Represents a queue belonging to a tenant.

Suggested Fields

Field	Purpose
<code>id</code>	Unique queue identifier
<code>tenant_id</code>	Tenant owning the queue
<code>name</code>	Queue name
<code>status</code>	Current queue status
<code>joined</code>	Whether users can join
<code>created_at</code>	Creation timestamp
<code>updated_at</code>	Last modification timestamp

Possible queue statuses:

ACTIVE

PAUSED

CLOSED

The final status model may be expanded if required.

15. Queue Ownership

Every queue must belong to exactly one tenant.

Tenant A

|

+---- Queue A1

+---- Queue A2

Tenant B

|

+---- Queue B1

+---- Queue B2

A queue must never belong to multiple tenants.

16. Queue Entry Table

Purpose

Represents one person's participation in a queue.

This is one of the most important tables in the system.

Suggested Fields

Field	Purpose
<code>id</code>	Unique queue-entry identifier
<code>tenant_id</code>	Tenant owning the entry
<code>queue_id</code>	Queue relationship

name	Person's name
phone	Person's phone number
email	Email where applicable
queue _numb er	Assigned queue number
statu s	Current queue-entry status
joine d_at	Time the person joined
calle d_at	Time the person was called
start ed_at	Time service started where applicable
compl eted_ at	Time service completed

<code>cancelled_at</code>	Time user cancelled
---------------------------	---------------------

<code>expired_at</code>	Time entry expired
-------------------------	--------------------

<code>created_at</code>	Record creation timestamp
-------------------------	------------------------------

<code>updated_at</code>	Last modification timestamp
-------------------------	--------------------------------

Additional configurable fields may be stored according to the final registration-field design.

17. Queue Entry Status

The queue entry lifecycle should support:

`WAITING`



`CALLED`



COMPLETED

Additional states may include:

CANCELLED

EXPIRED

NO_SHOW

The exact state-transition rules will be enforced by backend business logic.

18. Queue Number vs Queue Position

The system must distinguish between:

Queue Number

The identifier assigned to the queue entry.

Current Position

The person's current place among active waiting entries.

Example:

Original queue numbers:

Person A → #1

Person B → #2

Person C → #3

Person D → #4

After Person A completes:

Current active positions:

Person B → #1

Person C → #2

Person D → #3

The application should not rely on permanently rewriting historical identifiers merely to display the current position.

19. Recommended Position Strategy

The database should store a stable queue-entry identity and

calculate/display the current active position from the queue state.

Conceptually:

Stable Queue Entry

+

Queue Ordering

↓

Current Position

This avoids unnecessary destructive renumbering.

Example:

Entry ID	Original Number
----------	-----------------

A	1
---	---

B	2
---	---

C	3
---	---

D	4
---	---

After A is completed:

Active Entries:

B → Position 1

C → Position 2

D → Position 3

The stable queue-entry IDs remain unchanged.

20. Queue Ordering

The default queue ordering is:

First Come



First Served

The ordering should primarily use queue-entry creation/join order.

Example:

Joined At

A → 10:01

B → 10:02

C → 10:03

Queue:

A → #1

B → #2

C → #3

21. Admin Queue Reordering

An Organization Admin can manually change the order of a queue entry.

Example:

Before:

A → #1

B → #2

C → #3

D → #4

Admin prioritizes D:

After:

A → #1

D → #2

B → #3

C → #4

The database must preserve the resulting order safely.

The implementation may use an ordering/ranking mechanism rather than rewriting every queue-entry identifier.

22. Queue Ordering Field

A dedicated ordering mechanism should be considered.

Possible implementation:

position_order

or another backend-controlled ordering strategy.

The final implementation must support:

- Insert at position.
- Move entry.
- Remove entry.
- Reorder entries.
- Maintain deterministic ordering.
- Handle concurrent changes safely.

The exact mechanism will be finalized in [architecture.md](#).

23. DONE Operation

When an operator marks a queue entry as:

DONE

the database operation should:

Queue Entry



Status = COMPLETED

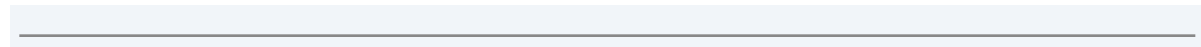


Completed timestamp recorded



Entry removed from active waiting set

The remaining active entries automatically move forward in their displayed positions.



24. Example DONE Flow

Before:

#1 A

#2 B

#3 C

#4 D

A is marked DONE.

Database state:

A → COMPLETED

B → WAITING

C → WAITING

D → WAITING

Displayed queue:

#1 B

#2 C

#3 D

No historical record should be destroyed.

25. User Cancellation

An end user can leave the queue.

The cancellation should:

1. Validate the queue entry.
2. Verify that cancellation is allowed.
3. Update the entry status.
4. Record cancellation time.
5. Record a queue event.
6. Remove the entry from the active queue.

7. Update remaining users.
8. Trigger the required notification/realtime event.

Example:

WAITING



CANCELLED

26. Cancellation Warning

The warning shown before cancellation is primarily a frontend concern.

The backend must still validate the cancellation request.

The user should be informed that:

Leaving the queue means losing the current queue position.

27. Queue Clearing

If multiple active entries are completed/cancelled/removed:

#1

#2

#3

#4

may become:

Queue Cleared

The database must preserve historical entries while marking them with their appropriate final state.

The system must not delete historical records simply because they are no longer active.

28. Queue Events Table

Purpose

Stores important events that happen to queue entries or queues.

This provides historical traceability.

Suggested Fields

Field

Purpose

<code>id</code>	Event identifier
<code>tenant_id</code>	Tenant
<code>queue_id</code>	Queue
<code>queue_entry_id</code>	Related queue entry where applicable
<code>event_type</code>	Type of event
<code>actor_user_id</code>	User/operator/admin responsible where applicable
<code>metadata</code>	Additional event information
<code>created_at</code>	Event timestamp

29. Queue Event Types

Potential events include:

QUEUE_JOINED

QUEUE_POSITION_CHANGED

QUEUE_CALLED

QUEUE_COMPLETED

QUEUE_CANCELLED

QUEUE_EXPIRED

QUEUE_NO_SHOW

QUEUE_REORDERED

QUEUE_PAUSED

QUEUE_RESUMED

QUEUE_CLOSED

QUEUE_CLEARED

The final event list may be adjusted during architecture design.

30. Why Queue Events Are Required

Queue events allow the system to understand what happened over time.

Example:

10:00 → Person joined

10:05 → Position changed

10:15 → Person called

10:20 → Person completed

This information can later support:

- Analytics.
 - Debugging.
 - Auditing.
 - Waiting-time calculations.
 - Operational reporting.
-

31. Estimated Waiting Time Data

The application needs historical service-duration information to estimate

waiting time.

A completed queue entry can provide:

Service Duration

=

Completed At - Service Start At

Where applicable.

Example:

Started: 10:10

Completed: 10:18

Service duration = 8 minutes

The system can use completed service durations to calculate an approximate average.

32. Average Service Time

The estimated waiting-time system should use actual completed service durations.

Example:

Previous durations:

8 minutes

10 minutes

7 minutes

9 minutes

Average:

$$(8 + 10 + 7 + 9) / 4$$

$$= 8.5 \text{ minutes}$$

33. Estimated Waiting-Time Calculation

For a simple FIFO queue:

Estimated Waiting Time

=

People Ahead

×

Average Service Time

Example:

Average service time = 8.5 minutes

Current position = #4

People ahead = 3

Estimated wait:

3×8.5

≈ 25.5 minutes

The displayed value may be rounded to a user-friendly approximation.

34. Insufficient Historical Data

When there are not enough completed entries to calculate a meaningful average, the system should not present a misleading precise estimate.

Possible behavior:

Estimated Wait:

Calculating...

or:

Estimated Wait:

Approximately 10 minutes

using a configured/default approach if one is defined.

The final fallback behavior will be decided during architecture/business-rule implementation.

35. Notification Table

Purpose

Represents a notification generated by the platform.

Suggested Fields

Field	Purpose
id	Notification identifier
tenant_id	Tenant
queue_entry_id	Related queue entry
type	Notification event type
message	Notification content/template reference
created_at	Creation timestamp

36. Notification Delivery Table

Because WhatsApp and SMS are both required, delivery attempts should be tracked separately.

Suggested Fields

Field	Purpose
id	Delivery identifier
notification_id	Notification relationship
channel	WhatsApp or SMS
status	Delivery state
provider_message_id	External provider reference where available
attempt_count	Number of attempts
sent_at	Sent timestamp

<code>delivered_at</code>	Delivery timestamp where available
---------------------------	------------------------------------

<code>failed_at</code>	Failure timestamp
------------------------	-------------------

<code>failure_reason</code>	Failure information where appropriate
-----------------------------	---------------------------------------

<code>created_at</code>	Creation timestamp
-------------------------	--------------------

<code>updated_at</code>	Last modification timestamp
-------------------------	-----------------------------

37. Notification Channels

The supported channels are:

WHATSAPP

SMS

Both channels are first-class notification channels.

SMS is not automatically considered only a fallback.

38. Notification Configuration

WhatsApp and SMS availability should be controlled through environment/configuration.

The database should store business-level notification configuration where required.

Provider credentials must NOT be stored as plain database configuration.

Secrets belong in secure environment/secret management.

39. Notification Status

Potential delivery states:

PENDING

PROCESSING

SENT

DELIVERED

FAILED

The final state model depends on the capabilities of the selected providers.

40. Notification Idempotency

The system should prevent accidental duplicate notifications.

A notification event should have a unique identity or idempotency mechanism.

Example:

TURN_AVAILABLE



Notification Created



WhatsApp



SMS

The same event should not unintentionally generate multiple duplicate notifications.

41. Audit Events Table

Purpose

Stores important administrative/security actions.

Potential events include:

ADMIN_GENERATED_QR

ADMIN_REORDERED_QUEUE

OPERATOR_MARKED_DONE

ADMIN_CHANGED_QUEUE

USER_CANCELLED_QUEUE

QUEUE_CREATED

QUEUE_UPDATED

QUEUE_DELETED

Suggested Fields

Field	Purpose
-------	---------

id	Audit event ID
----	----------------

tenan	Tenant
-------	--------

t_id

actor
_user
_id

Person performing
the action

action

Action name

entity_type

Affected entity

entity_id

Affected record

metadata

Relevant details

created_at

Timestamp

42. QR Code Data

The QR code is associated with a queue.

The system should not need to store the QR image itself in PostgreSQL unless there is a specific reason.

The database should store the required queue relationship/configuration.

Conceptually:

QR



Queue ID



Tenant ID

Only an authorized Organization Admin can generate the QR.

43. QR Security

A QR code must not provide unrestricted access to tenant data.

Scanning a QR should only identify the intended public queue-entry flow.

The backend must validate:

- Queue exists.
- Queue is active/open for joining.

- Queue belongs to the expected tenant.
 - QR identifier/token is valid.
 - User is allowed to join.
-

44. Configurable Registration Fields

The PRD allows queue-specific registration fields.

If configurable fields are implemented, the database may require additional configuration entities.

Potential concepts:

queue_form_fields

Possible properties:

field name

field type

required

display order

validation rules

Example:

Name → Text → Required

Phone → Phone → Required

Reference → Text → Optional

The exact implementation will be finalized based on MVP scope.

45. User Data Minimization

The database should store only information required for the queue/application.

Potential personal data:

- Name.
- Phone.
- Email.
- Organization-specific registration information.

Sensitive information should not be collected unnecessarily.

46. Data Retention

Historical queue records should be retained according to the final business/privacy policy.

The system must support future data-retention policies.

Potential policies:

30 days

90 days

1 year

Custom

The final retention period must be confirmed before production deployment if required by the client.

47. Data Deletion

Deleting active queue entries should not automatically mean permanently deleting historical records.

Example:

User leaves queue



Status = CANCELLED

The historical event remains available for appropriate operational/audit purposes.

Permanent deletion should follow the application's data-retention policy.

48. Database Constraints

The database should enforce appropriate constraints.

Examples:

- Required tenant relationship.
 - Required queue relationship.
 - Valid status values.
 - Unique tenant identifiers.
 - Unique queue identifiers where required.
 - Foreign-key relationships.
 - Valid timestamps.
 - Appropriate uniqueness constraints.
-

49. Foreign Key Relationships

Conceptually:

tenants

|

+---- users

|

+---- queues

|

+---- queue_entries

| |

+---- queue_events

|

+---- notifications

|

+---- notification_deliveries

Foreign keys should prevent orphaned records where appropriate.

50. Indexing Strategy

Indexes must support the application's most frequent queries.

Important candidates include:

Queues

tenant_id

status

Queue Entries

tenant_id

queue_id

status

created_at

Queue Events

tenant_id

queue_id

queue_entry_id

created_at

Notifications

tenant_id

queue_entry_id

created_at

Notification Deliveries

notification_id

channel

status

The final indexes will be validated using actual query patterns.

51. Active Queue Query

The system frequently needs to retrieve active waiting entries.

Conceptually:

SELECT active queue entries

WHERE

queue_id = current_queue

AND

status = WAITING

ORDER BY queue order

This query must be optimized because it is part of the core queue experience.

52. Queue Position Query

A user's displayed position should be derived from the active queue ordering.

Conceptually:

Current Position

=

Number of active entries ahead

+

1

The backend must calculate this consistently.

53. Concurrency

Queue operations can occur simultaneously.

Example:

Operator A → DONE Person #1

Operator B → DONE Person #2

User C → JOIN QUEUE

The database operations must maintain consistent queue state.

Transactions and appropriate locking/concurrency controls should be used where required.

54. Queue Number Assignment

Concurrency

When two people join simultaneously:

User A → JOIN

User B → JOIN

the system must not assign the same active queue position.

Example:

User A → #10

User B → #11

The final implementation must use database-safe transactional logic.

55. Admin Reordering Concurrency

If two administrators attempt to reorder the queue simultaneously, the system must prevent corrupted queue order.

The backend should:

1. Validate the current queue state.
 2. Apply the change transactionally.
 3. Commit the resulting order.
 4. Publish the final queue state.
-

56. Transaction Requirements

The following operations should be considered transactional:

Join Queue

Validate



Create Entry



Assign Ordering



Commit

Mark DONE

Validate



Update Entry



Record Event



Commit



Publish Real-Time Update

Admin Reorder

Validate



Reorder



Record Event



Commit



Publish Update

External notifications should generally be triggered after the database state is safely committed.

57. Database and Real-Time Separation

The database is the source of truth.

Real-time communication is a mechanism for notifying connected clients that the state has changed.

PostgreSQL



Authoritative State

Real-Time Layer



State Change Delivery

The real-time layer must not become the permanent source of queue truth.

58. Database and Notification Separation

Notifications should be generated from committed queue events.

Conceptually:

Queue Change



PostgreSQL Transaction



Commit



Notification Event



WhatsApp / SMS

A notification failure must not roll back a successfully completed queue operation.

59. Soft State vs Historical Data

Active queue state and historical information should be treated differently.

Active State

Used for:

- Current position.
- Current status.
- Current queue order.

Historical Data

Used for:

- Completed service durations.
- Queue events.
- Notification history.
- Audit history.

- Analytics.

Historical data should not be unnecessarily deleted when an entry leaves the active queue.

60. Database Security

Database security requirements include:

- Secure database credentials.
 - No credentials committed to Git.
 - Restricted database access.
 - Least-privilege database users.
 - Encrypted connections where applicable.
 - Tenant isolation.
 - Proper backups.
 - Secure production configuration.
-

61. Sensitive Data

The database may contain:

Name

Phone Number

Email

Queue Information

Organization Information

Access to these fields must be controlled.

Phone numbers and other personal information must not be exposed through public queue endpoints.

62. Public Queue Access

End users may access queue status without traditional authentication where required.

However, public access must be restricted to the user's own queue entry/status.

The system should use a secure, non-guessable identifier/token where a public status link is provided.

Example:

Public Queue Token



Specific Queue Entry



Allowed Status Information

A user must not be able to enumerate other queue entries by changing an ID.

63. Database Backup and Recovery

The production database must have a backup and recovery strategy.

The final infrastructure configuration should define:

- Backup frequency.
- Backup retention.
- Recovery procedure.
- Recovery testing.
- Disaster recovery requirements.

These details will be finalized during deployment planning.

64. Migration Strategy

Database schema changes must be managed through version-controlled migrations.

The project must not rely on manually modifying production database structures.

Every schema change should have:

Migration



Review



Test



Apply

65. Database Documentation Maintenance

Whenever database behavior changes:

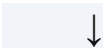
Implementation Change



Database Schema Review

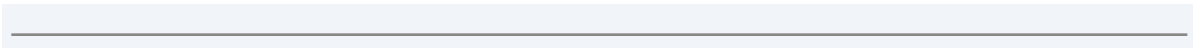


database.md Update



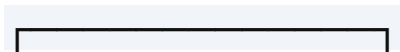
Architecture Review

The database documentation must remain synchronized with the implementation.

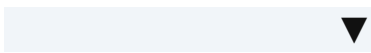
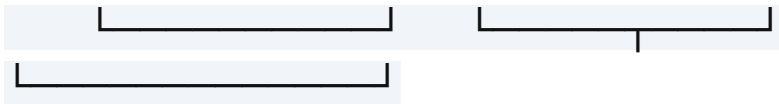


66. Initial Database Relationship Diagram





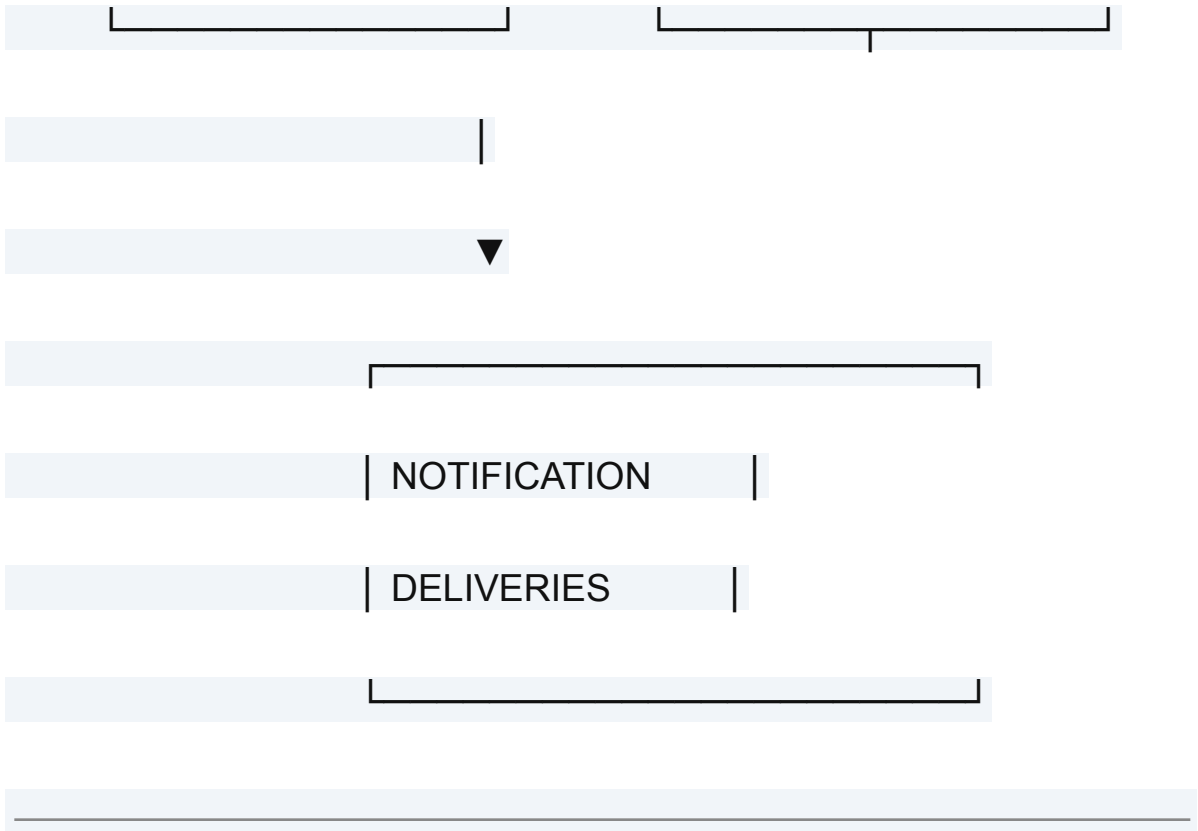
| USERS | | QUEUES | | AUDIT EVENTS |



| QUEUE ENTRIES |



| QUEUE EVENTS | | NOTIFICATIONS |



67. Core Data Flow

Tenant



Queue



User joins



Queue Entry created



Queue position determined



Queue Event recorded



Real-Time state published



Queue progresses



Operator marks DONE



Queue Entry becomes COMPLETED



Service duration recorded



Average service time updated



Remaining queue positions update



Notification event generated where required

68. Database Acceptance Criteria

The database design is considered ready when:

- PostgreSQL is confirmed.
- Multi-tenant strategy is defined.
- Tenant isolation is defined.
- Tenant relationships are defined.
- User roles are defined.
- Queue structure is defined.
- Queue-entry structure is defined.
- Queue states are defined.
- Queue ordering strategy is defined.
- Admin reordering is supported.
- DONE behavior is supported.
- User cancellation is supported.

- Queue clearing is supported.
 - Estimated waiting-time data is supported.
 - Queue events are stored.
 - WhatsApp notifications are supported.
 - SMS notifications are supported.
 - Notification delivery status is tracked.
 - Audit events are supported.
 - Appropriate indexes are defined.
 - Foreign keys are defined.
 - Constraints are defined.
 - Concurrency requirements are considered.
 - Transaction requirements are defined.
 - Data retention is considered.
 - Database security is defined.
 - Backup/recovery strategy is considered.
 - Migration strategy is defined.
-

69. Final Database Principles

The database must follow these principles:

Multi-Tenant

Tenant data must remain isolated.

Relational

PostgreSQL relationships and constraints should be used appropriately.

Consistent

Queue operations must maintain valid state.

Transactional

Critical queue operations must be atomic where required.

Auditable

Important queue and administrative events should be traceable.

Secure

Personal and tenant data must be protected.

Scalable

The schema must support multiple tenants and large numbers of queue entries.

Maintainable

Database structure and migrations must remain understandable.

Generic

The database must model queue management rather than a specific industry.

70. Final Database Model

The core database model is:

TENANT

|

└─ USERS

|

└─ QUEUES

| |

| └─ QUEUE ENTRIES

| |

| | └─ QUEUE EVENTS

| |

| | └─ NOTIFICATIONS

| |

| | └─ NOTIFICATION DELIVERIES

|

└─ AUDIT EVENTS

The database is intentionally centered around:

Tenant



Queue



Queue Entry



Queue State



Queue Events



Notifications

The model does not assume restaurants, tables, rooms, counters, doctors, desks, or other industry-specific resources.

END OF DATABASE DOCUMENT

